UNLOX

DIGITAL ELECTRONICS

Practice Assignment – Worked Solutions

Reference level: Problems are designed around the standard Digital Design topics covered in M. Morris Mano's *Digital Design*. They are original practice questions, not reproduced textbook questions.

Student Name	Thipperudra K T
Roll No.	4SM23EE037
Date	19 October 2026
Instructor	Kuldeep Sompura

Instructions

- Show important steps, Boolean simplifications, truth tables, K-maps, and timing diagrams where applicable.
- Clearly label inputs, outputs, states, select lines, and don't-care conditions.
- For design questions, give the final logic/circuit and a short explanation.
- Questions progress from basic practice to design-oriented problems.

Conventions. A' means NOT A; AB means A AND B; + means OR; ⊕ means XOR; ⊙ means XNOR. Σm(…) lists minterms; d(…) lists don't-cares. Bit 0 is the least significant bit (LSB). Flip-flops are positive-edge triggered unless stated.

Stage	Topic	Questions
1	Number systems and codes	1–15
2	Boolean algebra and K-maps	16–20
3	Combinational circuits	21–32
4	Flip-flops and counters	33–42
5	FSM and timing	43–48
6	Mixed design and verification	49–56

PART I – NUMBER SYSTEMS & DIGITAL CODES

Q1.

Convert 45, 173 and 625 from decimal to binary, octal and hexadecimal.

Repeated division by 2 (binary), then group bits in 3s (octal) and 4s (hex).

Decimal	Binary	Octal	Hex
45	101101 (32+8+4+1)	55	2D
173	10101101 (128+32+8+4+1)	255	AD
625	1001110001 (512+64+32+16+1)	1161	271

Check for 625: 001 001 110 001 → 1161 (octal); 0010 0111 0001 → 271 (hex).

Q2.

Convert 101101.101 (binary) to decimal and hexadecimal.

Integer part: 101101 = 32+8+4+1 = 45. Fraction: .101 = 1/2 + 1/8 = 0.625. **Decimal = 45.625**.

Hex: group outwards from the point: 0010 1101 . 1010 → **2D.A (hex)**.

Q3.

Convert 3A7 (hex) to binary and decimal.

3 = 0011, A = 1010, 7 = 0111 → **0011 1010 0111 = 1110100111**. Decimal = 3×256 + 10×16 + 7 = 768 + 160 + 7 = **935**.

Q4.

Perform 10110110 + 01101101 using binary addition and show all carries.

Bit position	7	6	5	4	3	2	1	0
Carry into bit	1	1	1	1	1	0	0	0
A (182)	1	0	1	1	0	1	1	0
B (109)	0	1	1	0	1	1	0	1
Sum	0	0	1	0	0	0	1	1

Final carry-out of bit 7 = 1, so the 9-bit result is 1 0010 0011.

Result = **1 0010 0011** = 291 (182 + 109 = 291 ✓). The 8-bit sum is 00100011 with a carry-out of 1.

Q5.

Perform 11001010 − 01011101 using 2's-complement subtraction.

Minuend = 11001010 (202); subtrahend = 01011101 (93).
1's complement of 01011101 = 10100010; add 1 → 2's complement = 10100011.

	1	1	0	0	1	0	1	0
+	1	0	1	0	0	0	1	1
= 1	0	1	1	0	1	1	0	1

The leading 1 is the carry-out and is discarded.

Carry-out = 1 (no borrow, result positive). **Result = 01101101 = 109** ✓ (202 − 93).

Q6.

Represent −37 in 8-bit signed-magnitude, 1's-complement and 2's-complement forms.

+37 = 00100101.

Form	Representation
Signed-magnitude	1 0100101 = **10100101** (sign bit 1, magnitude 37)
1's complement	invert all bits of 00100101 → **11011010**
2's complement	1's complement + 1 → **11011011**

Q7.

Range of an n-bit unsigned and an n-bit 2's-complement number; apply for n = 8 and n = 12.

Unsigned: 0 to $2^n - 1$. 2's complement: -2^{n-1} to $+(2^{n-1} - 1)$.

n	Unsigned	2's complement
8	0 to 255	−128 to +127
12	0 to 4095	−2048 to +2047

Q8.

Perform 3F2(hex) + A7C(hex) and 800(hex) − 2D5(hex).

Addition: 2 + C = E (no carry); F + 7 = 22 = 0x16 → write 6, carry 1; 3 + A + 1 = E. Result **E6E (hex)** (1010 + 2684 = 3694 ✓).

Subtraction: 0 − 5 → borrow: 16 − 5 = B; 0 − D − 1 → borrow: 16 − 14 = 2; 8 − 2 − 1 = 5. Result **52B (hex)** (2048 − 725 = 1323 ✓).

Q9.

Convert 0.6875 (decimal) and 19.625 (decimal) to binary.

0.6875 × 2 = 1.375 → 1; 0.375 × 2 = 0.75 → 0; 0.75 × 2 = 1.5 → 1; 0.5 × 2 = 1.0 → 1. So **$0.6875 = 0.1011_2$**.

19 = 10011; 0.625: 0.625×2 = 1.25 → 1; 0.25×2 = 0.5 → 0; 0.5×2 = 1.0 → 1 → .101. So **$19.625 = 10011.101_2$**.

Q10.

Explain signed overflow. Check 01110110 + 01011101 and 10010110 + 10100101.

Signed (2's-complement) overflow occurs when the true result is outside the representable range (−128…+127 for 8 bits). It happens only when two operands of the **same sign** give a result of the **opposite sign**, equivalently when carry into the MSB ≠ carry out of the MSB. Adding operands of opposite sign can never overflow.

#	Operands	Result (8-bit)	C_{in} MSB / C_{out} MSB	Overflow?
1	+118 + +93	11010011 (= −45, true sum +211)	1 / 0	**Yes** (pos + pos → neg)
2	−106 + −91	00111011 (= +59, true sum −197)	0 / 1	**Yes** (neg + neg → pos)

Q11.

Convert decimal 59 to BCD, Excess-3 and Gray code.

- **BCD:** each digit separately: 5 = 0101, 9 = 1001 → **0101 1001**.
- **Excess-3:** add 3 to each digit: 5+3 = 8 = 1000, 9+3 = 12 = 1100 → **1000 1100**.
- **Gray:** $59 = 111011_2$; G = B ⊕ (B >> 1) = 111011 ⊕ 011101 = **100110**.

Q12.

Convert Gray code 1101011 to binary, then back to Gray.

$B_6 = G_6 = 1$; each next bit $B_i = B_{i+1} \oplus G_i$: 1, 1⊕1 = 0, 0⊕0 = 0, 0⊕1 = 1, 1⊕0 = 1, 1⊕1 = 0, 0⊕1 = 1 → **binary = 1001101** (77).

Back to Gray: G = B ⊕ (B>>1) = 1001101 ⊕ 0100110 = **1101011** ✓ (original recovered).

Q13.

Design a 4-bit binary-to-Gray converter; derive the Boolean equation for each Gray output.

Input B3B2B1B0, output G3G2G1G0. The MSB is copied; every other Gray bit is the XOR of adjacent binary bits.

G3 = B3, G2 = B3 ⊕ B2, G1 = B2 ⊕ B1, G0 = B1 ⊕ B0.

Circuit: one wire and three 2-input XOR gates. Check: B = 0110 → G = 0, 0⊕1, 1⊕1, 1⊕0 = 0101 ✓.

Q14.

Why are 1010–1111 invalid BCD codes? How many invalid 4-bit combinations exist?

BCD encodes one decimal digit (0–9) in 4 bits. A 4-bit field has 2^4 = 16 combinations, but only 0000–1001 (0–9) represent digits. 1010 (10) through 1111 (15) would need two digits, so they are not BCD digits. **Invalid combinations = 16 − 10 = 6**. They are treated as don't-cares when designing BCD logic.

Q15.

A 6-bit counter: number of states and maximum unsigned value; repeat for signed 2's complement.

- States = 2^6 = **64** (same for either interpretation).
- Unsigned: maximum = $2^6 - 1$ = **63** (range 0…63).
- Signed 2's complement: maximum = $2^5 - 1$ = **+31** (range −32…+31).

PART II – BOOLEAN ALGEBRA & COMBINATIONAL CIRCUITS

Q16.

Simplify F = A'B + AB + AB' using Boolean algebra.

F = B(A' + A) + AB' = B + AB' = (B + A)(B + B') = **A + B**.

Q17.

Minimize F(A,B,C) = Σm(1,2,3,5,7) using a 3-variable K-map.

A \ BC	00	01	11	10
0	0	1	1	1
1	0	1	1	0

Group of four: m1, m3, m5, m7 (column C = 1) → **C**. Group of two: m2, m3 → **A'B**. **F = C + A'B**.

Q18.

Minimize F(A,B,C,D) = Σm(0,2,3,6,7,8,10,11,14,15) using a 4-variable K-map.

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	1	1
11	0	0	1	1
10	1	0	1	1

Octet: all cells with C = 1 (m2,3,6,7,10,11,14,15) → **C**. Quad of corners: m0, m2, m8, m10 → **B'D'**. **F = C + B'D'**.

Q19.

For F = AB + A'C + BC, use the consensus theorem to decide whether BC is redundant.

Consensus theorem: XY + X'Z + YZ = XY + X'Z. Taking X = A, Y = B, Z = C gives AB + A'C + BC = AB + A'C. The consensus term BC is therefore **redundant**, and **F = AB + A'C**.

Q20.

Implement F = (A+B)(C+D) using only NAND gates.

A + B = NAND(A', B') and C + D = NAND(C', D'); an AND is a NAND followed by an inverter (NAND with tied inputs).

A'=NAND(A,A) B'=NAND(B,B)
C'=NAND(C,C) D'=NAND(D,D) (4 gates)

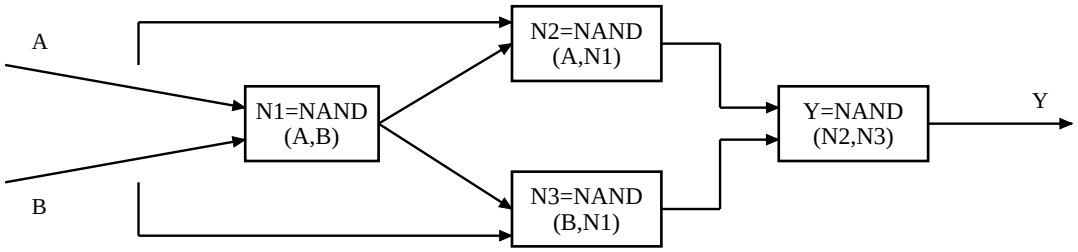
P = NAND(A',B') = A+B Q = NAND(C',D') = C+D (2 gates)

X = NAND(P,Q) F = NAND(X,X) = P·Q (2 gates)

Total = 8 two-input NAND gates. (Alternative: F = AC+AD+BC+BD in NAND–NAND form needs four 2-input NANDs plus a 4-input NAND.)

Q21.

Implement XOR using only 2-input NAND gates; give the minimum gate count.



N1 = (AB)'; N2 = (A·N1)' = (AB')'; N3 = (B·N1)' = (A'B)'; Y = (N2·N3)' = AB' + A'B = **A ⊕ B**. **Minimum = 4 NAND gates.**

Q22.

Truth table and equations for a half adder.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = A ⊕ B = A'B + AB'; Carry = AB (one XOR gate and one AND gate).

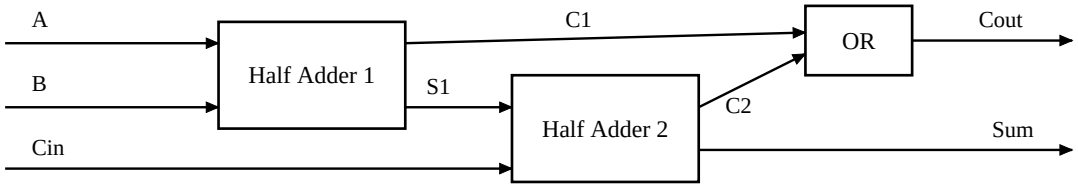
Q23.

Full adder from two half adders and an OR gate; derive Sum and Carry.

HA1: S1 = A ⊕ B, C1 = AB. HA2: Sum = S1 ⊕ C_{in}, C2 = S1·C_{in}. C_{out} = C1 + C2.

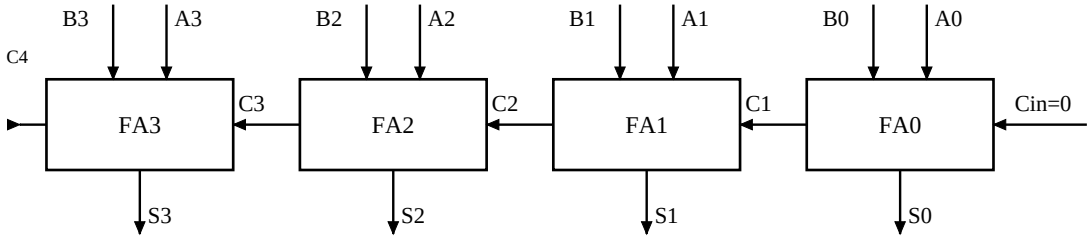
Sum = A ⊕ B ⊕ C_{in}; C_{out} = AB + C_{in}(A ⊕ B) = majority(A,B,C_{in}).

A	B	Cin	S1	C1	Sum	C2	Cout
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	1	0	0	1	1
1	0	0	1	0	1	0	0
1	0	1	1	0	0	1	1
1	1	0	0	1	0	0	1
1	1	1	0	1	1	0	1



Q24.

4-bit ripple-carry adder using full adders; explain carry propagation delay.



Inputs A3A2A1A0, B3B2B1B0 and C_{in} = C0 (0 for plain addition); outputs S3…S0 and carry-out C4. Each FA's carry-out feeds the next FA's carry-in.

Carry propagation delay: bit i cannot produce a valid sum until carry C_i arrives. In the worst case the carry ripples through all stages, so $t_{total} \approx n \cdot t_{carry}$ ($\approx 4 \cdot t_{carry} + t_{sum}$ here). Delay grows linearly with word length; it can be cut using a carry-lookahead adder with generate $G_i = A_i B_i$ and propagate $P_i = A_i \oplus B_i$.

Q25.

Design a 4-to-1 multiplexer using basic gates; write its output equation.

Data inputs I0–I3, select lines S1, S0, output Y.

Y = S1'S0'·I0 + S1'S0·I1 + S1S0'·I2 + S1S0·I3

Gates: 2 inverters (S1', S0'), four 3-input AND gates (one per data input), one 4-input OR gate.

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Q26.

Implement F(A,B,C) = Σm(1,2,5,7) with a 4-to-1 MUX using A and B as select inputs.

With S1 = A, S0 = B, the data input of each pair of minterms is a function of C:

AB	Minterms	F values (C=0, C=1)	Data input
00	m0, m1	0, 1	I0 = C
01	m2, m3	1, 0	I1 = C'
10	m4, m5	0, 1	I2 = C
11	m6, m7	0, 1	I3 = C

Connect I0 = C, I1 = C' (one inverter), I2 = C, I3 = C, S1 = A, S0 = B; Y = F.

Q27.

Design a 3-to-8 decoder with an enable input; write all outputs.

Inputs A2A1A0, enable E, outputs Y0…Y7 (active-high). Each output is a minterm ANDed with E:

Y0 = E·A2'A1'A0' Y1 = E·A2'A1'A0 Y2 = E·A2'A1A0' Y3 = E·A2'A1A0

Y4 = E·A2A1'A0' Y5 = E·A2A1'A0 Y6 = E·A2A1A0' Y7 = E·A2A1A0

Implementation: eight 4-input AND gates plus three inverters. When E = 0 all outputs are 0; when E = 1 exactly one output (selected by A2A1A0) is 1. (With NAND gates the outputs become active-low.)

Q28.

Why does binary-to-BCD conversion generally require logic rather than direct wiring?

Binary values 0–9 have identical binary and BCD codes, but for 10 and above the BCD code uses a separate 4-bit digit per decimal place and so is not a bit-for-bit rearrangement (binary 1010 = BCD 0001 0000). Carry from the units digit into the tens digit occurs at 10 rather than 16, i.e. 6 must be added to each digit that exceeds 9. Because this relationship is non-linear and depends on the whole input value, it needs arithmetic/combinational logic (e.g. the shift-and-add-3 "double dabble" method or ROM/adders), not simple wiring.

Q29.

Design a 2-bit magnitude comparator; derive A>B, A=B and A<B.

Inputs A = A1A0, B = B1B0. Let $x_i = A_i \odot B_i$ (XNOR, 1 when bits are equal).

A = B : (A=B) = x1 · x0

A > B : (A>B) = A1·B1' + x1 · A0·B0'

A < B : (A<B) = A1'·B1 + x1 · A0'·B0

The MSBs decide unless they are equal, in which case the LSBs decide. Exactly one of the three outputs is 1 for every input combination.

Q30.

One of eight sensor inputs must be routed to one output: choose a circuit and the number of select lines.

Use an **8-to-1 multiplexer**. Number of select lines = $\log_2 8$ = **3** (S2, S1, S0). Y = Σ (minterm of S2S1S0 · I_k), k = 0…7.

Q31.

Design a 4-bit parity generator and a 5-bit even-parity checker.

Generator (even parity): P = D3 ⊕ D2 ⊕ D1 ⊕ D0 (three XOR gates), which makes the total number of 1s in D3D2D1D0P even. (For odd parity use XNOR: P = (D3⊕D2⊕D1⊕D0)'.)

Checker: E = D3 ⊕ D2 ⊕ D1 ⊕ D0 ⊕ P (four XOR gates). E = 0 → parity OK (even number of 1s); E = 1 → single-bit error detected.

D3D2D1D0	P	Received (D+P)	E
1011	1	10111	0 (OK)
1011	1	10101 (bit flipped)	1 (error)

Q32.

Explain how don't-care conditions are used in K-map minimization; give a small example.

A don't-care (X or d) is an input combination that never occurs (or whose output is irrelevant). In a K-map you may treat each X as 1 if that lets you form a larger group, and as 0 otherwise; X cells never have to be covered. Larger groups mean fewer literals.

Example: F(A,B,C) = Σm(1,3,5) + d(7).

A \ BC	00	01	11	10
0	0	1	1	0
1	0	1	X	0

Without the don't-care: F = A'C + B'C (two groups of 2). With d7 taken as 1, the group m1, m3, m5, m7 gives **F = C**, a much simpler circuit.

PART III – SEQUENTIAL CIRCUITS

Q33.

Difference between combinational and sequential circuits; two examples each.

	Combinational	**Sequential**
Output depends on	present inputs only	present inputs and stored past state
Memory	none	yes (flip-flops / latches)
Clock	not required	usually clocked
Examples	adder, multiplexer (also decoder, comparator)	flip-flop, counter (also register, FSM)

Q34.

Compare SR, JK, D and T flip-flops.

FF	**Inputs**	**Characteristic equation / next state**	**Typical application**
SR	S, R	Q+ = S + R'Q (SR = 0 forbidden); S=R=0 hold, S=1 set, R=1 reset	basic latch, debouncing
JK	J, K	Q+ = JQ' + K'Q; 00 hold, 01 reset, 10 set, 11 toggle	counters, general-purpose
D	D	Q+ = D	registers, data storage, shift registers
T	T	Q+ = T ⊕ Q; T=0 hold, T=1 toggle	binary counters, frequency dividers

Q35.

Characteristic and excitation tables of a JK flip-flop; toggle condition.

J	**K**	**Q**	**Q+**
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q	**Q+**	**J**	**K**
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Toggle condition: J = K = 1 → Q+ = Q', i.e. the output complements on every active clock edge.

Q36.

A D flip-flop has D = Q'. Starting with Q = 0, find Q for six clock edges.

Clock edge	0 (start)	1	2	3	4	5	6
Q	0	1	0	1	0	1	0

Because D = Q', Q toggles every edge, so the circuit divides the clock frequency by 2.

Q37.

Synchronous 2-bit up counter using T flip-flops; T equations.

Q1	Q0	Q1+	Q0+	T1	T0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

T0 = 1, T1 = Q0. Both flip-flops share the same clock. Sequence 00 → 01 → 10 → 11 → 00.

Q38.

3-bit synchronous binary up counter: state sequence and T equations.

Sequence: 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000. A bit toggles when all lower bits are 1:

T0 = 1, T1 = Q0, T2 = Q1·Q0 (one AND gate). Check: 011 → T2T1T0 = 111 → next 100 ✓.

Q39.

Design a MOD-6 counter: valid/unused states and recovery.

Valid states 000…101 (0–5); **unused states: 110 and 111**. Using D flip-flops, make unused states go to 000 so the counter is self-correcting.

Present Q2Q1Q0	Next Q2+Q1+Q0+
000	001
001	010
010	011
011	100
100	101
101	000 (rollover)
110 (unused)	000 (recovery)
111 (unused)	000 (recovery)

D2 = Q2'Q1Q0 + Q2Q1'Q0' (m3 + m4)

D1 = Q2'(Q1 ⊕ Q0) (m1 + m2)

D0 = Q0'(Q2Q1)' (m0 + m2 + m4)

Recovery: any power-up/noise state in {110, 111} loads 000 on the next clock. An alternative is to detect 110 and use the flip-flops' synchronous clear.

Q40.

Ripple vs synchronous counters.

	Ripple (asynchronous)	Synchronous
Clocking	each FF clocked by the previous FF output	all FFs share one clock
Propagation delay	accumulates: $n \times t_{pd}$	one t_{pd} + gate delay; independent of n
Max frequency	low; limited by cumulative delay	high

	Ripple (asynchronous)	**Synchronous**
Hardware	simple, fewer gates	extra logic (AND gates) for T/J-K inputs
Glitches	decoding glitches on intermediate states	clean transitions
Use	low-speed frequency division	high-speed counting, state machines

Q41.

Bidirectional shift register: control signal and data path.

4-bit register Q3Q2Q1Q0 with a 2-to-1 MUX at each D input. Control signal **R/L** (1 = shift right, 0 = shift left); serial inputs SIR (into Q3) and SIL (into Q0).

D3 = R·SIR + R'·Q2 D2 = R·Q3 + R'·Q1

D1 = R·Q2 + R'·Q0 D0 = R·Q1 + R'·SIL

Data path: right shift Q3 → Q2 → Q1 → Q0 (Q0 is the serial out); left shift Q0 → Q1 → Q2 → Q3. All flip-flops are clocked together.

Q42.

4-bit SIPO register starts at 0000; shift in 1,0,1,1.

Serial bit enters Q0 and shifts toward Q3 (register written Q3Q2Q1Q0):

Clock	**Serial in**	**Q3Q2Q1Q0**
0 (start)	–	0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011

After 4 clocks the parallel output is 1011 (the first bit entered ends up in Q3). If the register shifts right instead (input into Q3), the contents are 1000, 0100, 1010, 1101.

Q43.

Overlapping sequence detector for 1011: state diagram and output.

Mealy machine, input X, output Z. States: S0 = no progress, S1 = "1", S2 = "10", S3 = "101".

Present	**X = 0 → next / Z**	**X = 1 → next / Z**
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S0 / 0	S3 / 0
S3	S2 / 0	S1 / **1**

State diagram (arrows): S0 –1/0→ S1; S1 –0/0→ S2; S2 –1/0→ S3; S3 –1/1→ S1 (detect, last "1" overlaps as start of next pattern); S3 –0/0→ S2 (suffix "10"); S0 –0/0→ S0; S1 –1/0→ S1; S2 –0/0→ S0.

Output condition: Z = 1 when in S3 and X = 1. With assignment S0 = 00, S1 = 01, S2 = 10, S3 = 11 (Q1Q0) and D flip-flops: **D1 = Q0X' + Q1Q0'X, D0 = X, Z = Q1Q0X**.

Q44.

Convert the detector to a Moore machine; explain output timing.

Add a state S4 = "1011 detected". Output depends only on the state: Z = 1 in S4.

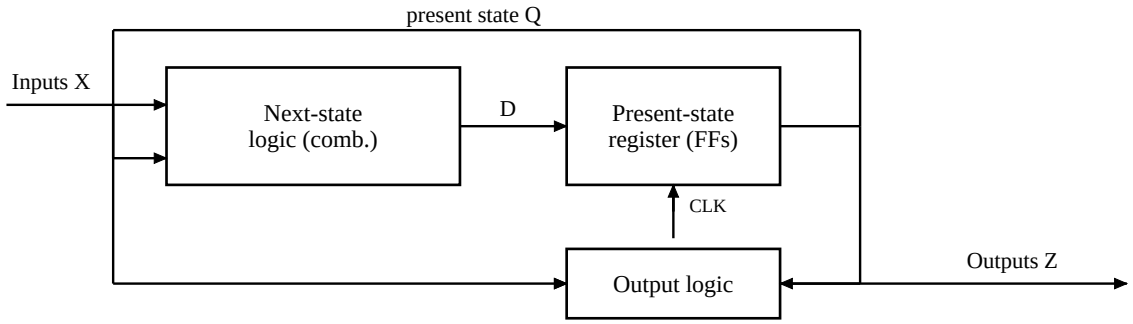
Present	**X = 0 → next**	**X = 1 → next**	**Z**
S0	S0	S1	0

Present	X = 0 → next	X = 1 → next	Z
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S4	0
S4	S2	S1	**1**

Timing difference: in the Mealy design Z = f(state, X) is asserted during the same cycle as the final input bit and can change with X (asynchronous glitches possible). In the Moore design Z = f(state) changes only after the clock edge that enters S4, so it appears **one clock cycle later**, but it is stable and glitch-free. The Moore machine needs one more state (5 vs 4).

Q45.

Generic synchronous FSM block diagram.



- **Present-state register:** flip-flops that store the current state; updated on every active clock edge.
- **Next-state logic:** combinational function of present state and inputs, producing the D (or J/K/T) inputs.
- **Output logic:** combinational function of state only (Moore) or state and inputs (Mealy).

Q46.

FSM with S0,S1,S2: X=1 advances S0→S1→S2→S0, X=0 holds. State table and assignment.

Assignment (binary): **S0 = 00, S1 = 01, S2 = 10** (11 unused, treated as don't-care).

Present Q1Q0	X = 0 → next	X = 1 → next
S0 (00)	S0 (00)	S1 (01)
S1 (01)	S1 (01)	S2 (10)
S2 (10)	S2 (10)	S0 (00)
11 (unused)	X	X

Using D flip-flops and the don't-cares in the K-map: **D1 = Q1X' + Q0X**, **D0 = Q1'Q0'X + Q0X'**. In unused state 11, X = 1 leads to S2 (10), so the machine recovers once X = 1 is applied.

Q47.

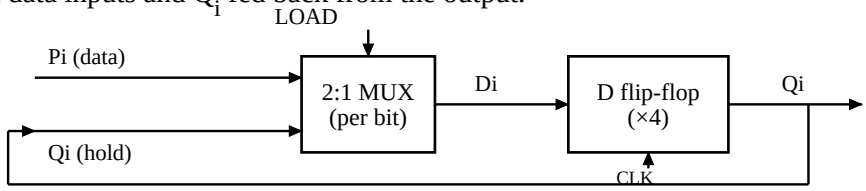
Setup time, hold time, clock-to-Q delay and metastability.

- **Setup time (t_{su}):** minimum time the data input must be stable *before* the active clock edge.
- **Hold time (t_h):** minimum time the data input must stay stable *after* the active clock edge.
- **Clock-to-Q delay (t_{cq}):** time from the active clock edge until the output Q becomes valid. It limits the clock period: $T_{clk} \geq t_{cq} + t_{logic} + t_{su}$.
- **Metastability:** if data changes within the setup/hold window (e.g. asynchronous input), the flip-flop output may hover at an invalid level and settle to 0 or 1 after an unpredictable time. It is reduced by synchronizers (two or more flip-flops in series) and by meeting timing.

Q48.

4-bit register with parallel load (LOAD=1) and hold (LOAD=0) using a 2-to-1 MUX.

For each bit i (0…3) place a 2-to-1 MUX in front of a D flip-flop: $\mathbf{D_i = LOAD \cdot P_i + LOAD' \cdot Q_i}$, with P_i the parallel data inputs and Q_i fed back from the output.



LOAD = 1: $Q_i+ = P_i$ (parallel load on the clock edge). LOAD = 0: $Q_i+ = Q_i$ (hold).

PART IV – MIXED DESIGN & VERIFICATION PRACTICE

Q49.

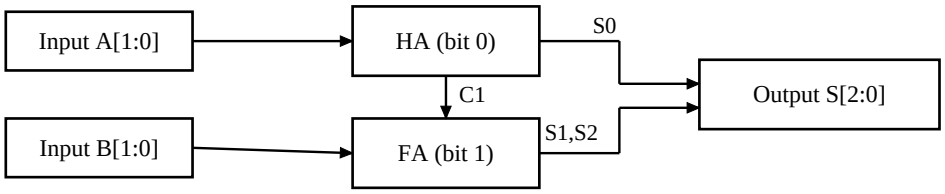
System that accepts two 2-bit numbers and produces their sum.

Inputs A = A1A0 and B = B1B0; output S = S2S1S0 (3 bits, max 3 + 3 = 6).

S0 = A0 ⊕ B0 C1 = A0·B0 (half adder)

S1 = A1 ⊕ B1 ⊕ C1 C2 = A1B1 + C1(A1 ⊕ B1) (full adder)

S2 = C2



Data flow: operands → bit-0 half adder → carry C1 → bit-1 full adder → {C2, S1, S0}. Optionally register inputs/outputs for synchronous operation.

Q50.

Counter with sequence 0,1,3,2,6,7,5,4, then repeat: state table and flip-flop implementation.

This is a 3-bit Gray-code counter. Use T flip-flops, T = Q ⊕ Q+.

Present Q2Q1Q0	Decimal	Next Q2+Q1+Q0+	T2 T1 T0
000	0	001	0 0 1
001	1	011	0 1 0
011	3	010	0 0 1
010	2	110	1 0 0
110	6	111	0 0 1
111	7	101	0 1 0
101	5	100	0 0 1
100	4	000	1 0 0

T0 = (Q2 ⊕ Q1 ⊕ Q0)' (1 when parity of present state is even)

T1 = Q0·(Q2 ⊙ Q1) = Q0(Q2Q1 + Q2'Q1')

T2 = Q0'·(Q2 ⊕ Q1)

All 8 states are used, so there are no unused states. Three T flip-flops with a common clock implement the counter.

Q51.

Alarm when at least three of four inputs are TRUE.

Inputs ABCD	Output
0111, 1011, 1101, 1110, 1111	1
all other combinations (11 of 16)	0

Minterms 7, 11, 13, 14, 15. K-map pairs give: **F = ABC + ABD + ACD + BCD** (four 3-input ANDs into a 4-input OR). Check: 1110 → ABC = 1 ✓; 1100 → no term = 1 → 0 ✓.

Q52.

4-bit one-hot to 2-bit binary encoder; assumptions for invalid inputs.

Inputs I3I2I1I0 (exactly one is 1), outputs Y1Y0, plus a valid flag V.

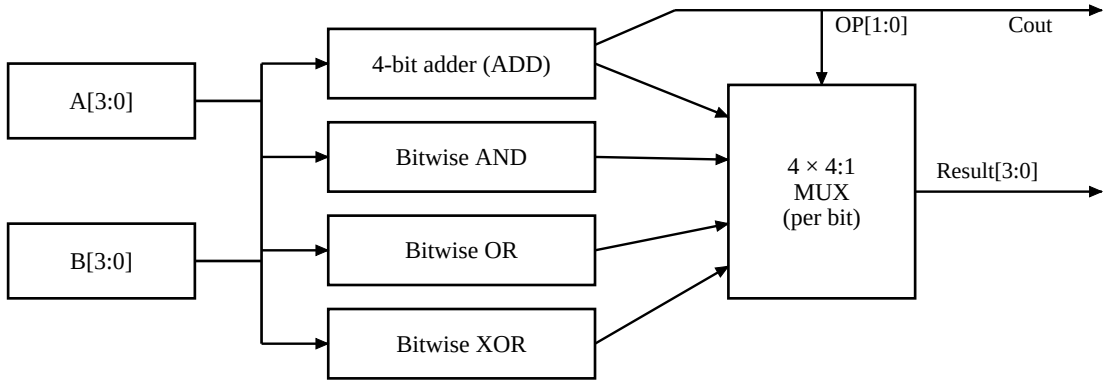
I3 I2 I1 I0	Y1 Y0
0 0 0 1	0 0
0 0 1 0	0 1
0 1 0 0	1 0
1 0 0 0	1 1

Y1 = I3 + I2, Y0 = I3 + I1 (I0 needs no gate: it is the all-zero code). **V = I3 + I2 + I1 + I0** distinguishes the valid code 0001 from the invalid input 0000 (both give Y = 00).

Assumptions: all-zero and multiple-1 patterns are invalid (don't-care in the design). With these equations multiple-hot inputs give an OR of the codes; to resolve them, use a priority encoder (highest index wins) and flag errors when more than one input is 1.

Q53.

4-bit ALU concept with ADD, AND, OR, XOR; block-level architecture.



OP[1:0]	Operation	Per-bit result
00	ADD	A + B (ripple/lookahead adder, Cout flag)
01	AND	$A_i \cdot B_i$
10	OR	$A_i + B_i$
11	XOR	$A_i \oplus B_i$

A 4-to-1 MUX per bit selects the active unit through OP[1:0]. Optional flags: carry-out (from the adder), zero (NOR of result bits).

Q54.

Overlapping detector for the pattern 110; why overlap matters.

Mealy FSM: S0 = no progress, S1 = "1", S2 = "11".

Present	X = 0 → next / Z	X = 1 → next / Z
S0	S0 / 0	S1 / 0
S1	S0 / 0	S2 / 0
S2	S0 / **1**	S2 / 0

Encoding S0 = 00, S1 = 01, S2 = 10 (D flip-flops): **D1 = X(Q1 + Q0), D0 = Q1'Q0'X, Z = Q1X'**.

Why overlap matters: an overlapping detector must reuse the tail bits of a detected (or partially matched) sequence as the head of the next one instead of restarting. For example "1110" must still detect after extra 1s

(S2 stays in S2 on X = 1), and in a pattern such as 1011 the final "1" starts a new match. Note that 110 itself has no proper suffix equal to a prefix (its suffixes "0" and "10" differ from its prefixes "1" and "11"), so for this pattern overlapping and non-overlapping detection behave identically; the overlap handling that matters here is that a run of extra 1s (S2 → S2) is not lost.

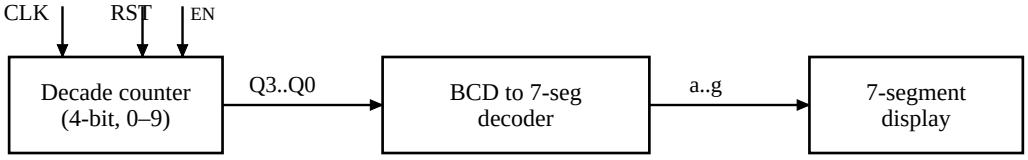
Q55.

Verification checks for a 4-bit synchronous counter.

- **Reset:** asserting reset (sync/async as designed) forces Q = 0000 regardless of the current state; Q stays 0000 while reset is held; counting starts correctly after release.
- **Every transition:** from each state N (0…15) the next state is N + 1 (mod 16) when enabled; check all 16 transitions with a self-checking testbench (expected value = previous + 1).
- **Hold/enable:** with enable = 0 the state is unchanged over several clocks.
- **Rollover:** 1111 → 0000 and carry-out/terminal-count asserts only at 1111.
- **Illegal-state recovery:** force or initialise unused/illegal states (for a MOD-N counter, states ≥ N) and verify the counter returns to a valid state within one clock.
- **Timing:** outputs change only on the active clock edge; no glitch-induced false counts; setup/hold met at the specified clock frequency.
- **Coverage:** all states and transitions visited; random reset/enable sequences; compare against a reference model.

Q56.

Mini project: 4-bit counter + decoder + seven-segment interface for a 0–9 counter.



- **Counter:** synchronous 4-bit counter with enable and reset; counts 0000…1001 and returns to 0000 (sync clear when Q = 1001: next = 0000). Illegal states 1010–1111 return to 0000.
- **Decoder:** BCD → seven segments a–g (common-cathode, 1 = on); codes 1010–1111 blank the display or are don't-cares.
- **Interface:** segment outputs drive the display through current-limiting resistors (one per segment); the clock is divided down to about 1 Hz to be visible.

Digit	Q3Q2Q1Q0	Segments ON (abcdefg)
0	0000	a b c d e f
1	0001	b c
2	0010	a b d e g
3	0011	a b c d g
4	0100	b c f g
5	0101	a c d f g
6	0110	a c d e f g
7	0111	a b c
8	1000	a b c d e f g
9	1001	a b c d f g

Major verification cases:

- Reset to 0, then count $0 \rightarrow 1 \rightarrow \ldots \rightarrow 9 \rightarrow 0$ (rollover) with correct display of each digit.
- Decoder: all ten valid codes produce the table above; invalid codes 1010–1111 give a safe blank/defined output.
- Enable = 0 holds the display; reset works at any count.
- Illegal-state forcing recovers within one clock; no glitches/ghost digits on the segments.
- Timing: counter-to-decoder delay is within one clock period.